

Indian Institute of Technology Madras

BS Electronic Systems

Signal Lab

Web-Based Signal Generator and Analyzer

Signal Processing Project Report

Author: Anjishnu Saw

Roll Number: 23f3000040

Project Code: SP24

Submission Date: 20-09-2026

Instructor: Vishal Raj

Contents

Abstract	3
1 Introduction	4
1.1 Motivation	4
1.2 Scope of the Project	4
2 Problem Statement	6
3 Goals and Objectives	7
3.1 Primary Goal	7
3.2 Specific Objectives	7
4 Methodology	8
4.1 System Overview	8
4.2 Signal Processing Techniques Used	8
4.2.1 Signal Generation	8
4.2.2 Signal Statistics	9
4.2.3 FFT-Based Frequency Analysis	9
4.2.4 Windowing	10
4.2.5 Short-Time Fourier Transform	10
4.3 Theoretical Background	11
5 Implementation Details	12
5.1 Tools and Technologies	12
5.2 System Architecture	12
5.2.1 Generator Module	12
5.2.2 DSP Module	12
5.2.3 Audio Module	13
5.2.4 React Components	13
5.3 Key Algorithms / Pseudocode	13
5.3.1 Signal Generation	13
5.3.2 FFT Spectrum	13
5.3.3 STFT	14
5.4 User Interface	14
5.4.1 Generator Panel	14
5.4.2 Analyzer Panel	15
6 Problems Faced and Solutions	16
6.1 Problem 1: Component Import Resolution	16
6.2 Problem 2: FFT and Spectrum Scaling	16
6.3 Problem 3: Sampling Rate Handling	16

6.4	Problem 4: Audio Playback and Export	16
6.5	Problem 5: Plot Readability	16
7	Results and Discussion	17
7.1	Experimental Setup	17
7.2	Quantitative Results	18
7.3	Application Screenshots	18
7.3.1	Complete Application Interface	18
7.3.2	Signal Generation and Time-Domain Analysis	19
7.3.3	Frequency-Domain Analysis	19
7.3.4	Windowing Analysis	20
7.3.5	Chirp and STFT Analysis	21
7.3.6	Audio Upload and Analysis	21
7.3.7	Interactive Plot View	22
7.4	Qualitative Results	23
7.4.1	Time-Domain Analysis	23
7.4.2	Frequency-Domain Analysis	23
7.4.3	STFT Analysis	23
7.4.4	Audio Analysis	23
7.5	Discussion	24
8	Conclusions	25
8.1	Summary of Contributions	25
8.2	Limitations	25
8.3	Future Work	25
9	Individual Contribution	26
10	Annexure	27
10.1	Video Demonstration	27
10.2	Source Code	27
10.3	Live Deployment	28
	References	29

Abstract

Signal Lab is a browser-based signal generator and analyzer developed for the Signal Processing project. The application generates sine, square, triangle, chirp, and sinc signals and provides interactive time-domain, frequency-domain, and time-frequency analysis. The frequency-domain analyzer uses a radix-2 FFT with selectable Rectangular, Hann, Hamming, and Blackman windows. The implementation uses coherent-gain correction and one-sided amplitude scaling for the displayed spectrum. The STFT uses a 1024-sample window and a 256-sample hop. The application also accepts local audio files, preserves their sampling rate, converts multi-channel input to mono, and sends the decoded signal through the same analysis pipeline. Browser playback and PCM16 WAV export are also provided. Signal statistics include RMS, peak amplitude, and DC mean. The complete system is implemented in React and JavaScript using Vite, HTML Canvas, and the Web Audio API. The application was tested using analytically predictable sine, square, and triangle signals, frequency-domain tests, windowing tests, chirp spectrograms, and uploaded audio.

Chapter 1

Introduction

Signal processing concepts are easier to understand when the effect of each operation can be observed directly. Signal Lab provides a compact web interface for generating signals and studying their time and frequency characteristics.

The project follows the SP24 requirement of a web-based signal generator and analyzer. The application combines signal generation, FFT analysis, STFT analysis, visualization, audio input, playback, and WAV export in one client-side system.

1.1 Motivation

The main motivation was to build a practical environment for observing basic DSP concepts without depending on external desktop software.

The application provides:

- Direct generation of standard signals.
- Time-domain visualization.
- Frequency-domain visualization.
- Windowing comparison.
- Time-frequency analysis.
- Analysis of real audio files.

1.2 Scope of the Project

In scope:

- Digital signal generation.
- Time-domain visualization.
- FFT-based frequency analysis.
- Windowed spectral analysis.
- STFT and spectrogram generation.
- Signal statistics.
- Local audio-file decoding.

- Browser playback.
- WAV export.

Out of scope:

- FIR or IIR filter design.
- Hardware signal acquisition.
- Real-time microphone DSP.
- Wavelet analysis.
- Advanced signal classification.

Chapter 2

Problem Statement

A signal generator and analyzer should provide a simple way to create known signals and observe their properties in both time and frequency domains. A single static plot is not sufficient for signals whose frequency changes with time.

The project therefore implements a web application that generates known signals, computes their signal statistics, obtains their frequency spectra using the FFT, and obtains time-frequency representations using the STFT. The same analysis pipeline is also used for uploaded audio.

The system operates on discrete-time samples. Generated signals use a default sampling frequency of 44.1 kHz. Uploaded audio uses its decoded sampling frequency.

Chapter 3

Goals and Objectives

3.1 Primary Goal

To implement a browser-based signal generator and analyzer for interactive study of fundamental DSP concepts.

3.2 Specific Objectives

- Generate sine, square, triangle, chirp, and sinc signals.
- Compute RMS, peak, and DC mean.
- Implement a radix-2 FFT for spectral analysis.
- Implement Rectangular, Hann, Hamming, and Blackman windows.
- Display a one-sided amplitude spectrum.
- Implement STFT-based spectrogram analysis.
- Support uploaded audio with its actual sampling rate.
- Provide browser playback and WAV export.
- Validate generated signals against theoretical expectations.
- Deploy the application as a web application.

Chapter 4

Methodology

4.1 System Overview

The complete processing flow is shown in Figure 4.1.

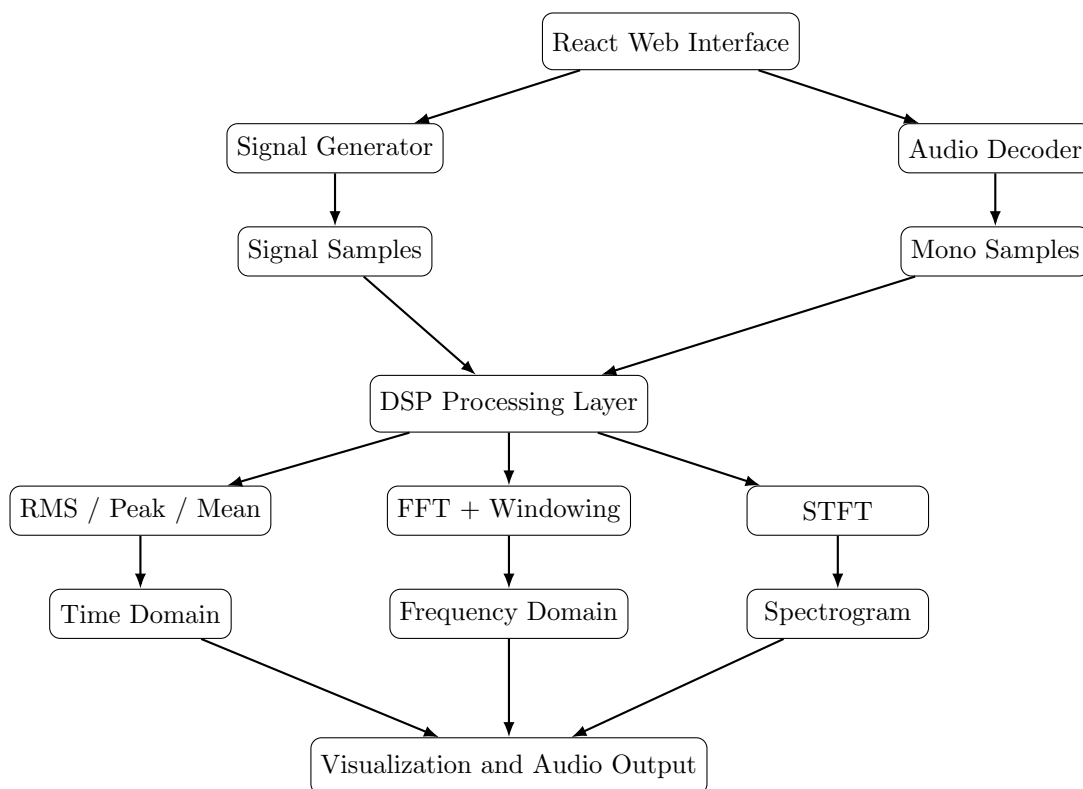


Figure 4.1: System architecture of Signal Lab

4.2 Signal Processing Techniques Used

4.2.1 Signal Generation

The application generates all signals directly as discrete-time samples.

For a sine wave:

$$x[n] = A \sin \left(2\pi f \frac{n}{F_s} \right) \quad (4.1)$$

where A is amplitude, f is frequency, and F_s is the sampling frequency.

The square wave is generated using a phase threshold:

$$x[n] = \begin{cases} A, & \phi[n] < D \\ -A, & \phi[n] \geq D \end{cases} \quad (4.2)$$

where D is the duty cycle represented as a fraction of one period.

The triangle wave is generated as:

$$x[n] = A \frac{2}{\pi} \sin^{-1} \left(\sin \left(2\pi f \frac{n}{F_s} \right) \right) \quad (4.3)$$

For a linear chirp, the instantaneous frequency changes from f_0 to f_1 over duration T :

$$x(t) = A \sin \left[2\pi \left(f_0 t + \frac{K}{2} t^2 \right) \right] \quad (4.4)$$

where

$$K = \frac{f_1 - f_0}{T} \quad (4.5)$$

The sinc signal uses:

$$\text{sinc}(x) = \begin{cases} 1, & x = 0 \\ \frac{\sin(\pi x)}{\pi x}, & x \neq 0 \end{cases} \quad (4.6)$$

4.2.2 Signal Statistics

For a signal containing N samples, the mean is:

$$\mu = \frac{1}{N} \sum_{n=0}^{N-1} x[n] \quad (4.7)$$

The RMS value is:

$$x_{\text{RMS}} = \sqrt{\frac{1}{N} \sum_{n=0}^{N-1} x^2[n]} \quad (4.8)$$

The peak value is:

$$x_{\text{peak}} = \max_n |x[n]| \quad (4.9)$$

4.2.3 FFT-Based Frequency Analysis

The DFT is defined as:

$$X[k] = \sum_{n=0}^{N-1} x[n] e^{-j2\pi kn/N} \quad (4.10)$$

The application uses an iterative radix-2 Cooley-Tukey FFT instead of computing the DFT directly.

The FFT size is selected as the next power of two greater than or equal to the input length.

The frequency corresponding to bin k is:

$$f_k = \frac{kF_s}{N} \quad (4.11)$$

The frequency resolution is:

$$\Delta f = \frac{F_s}{N} \quad (4.12)$$

Only the positive-frequency half is displayed.

For the one-sided amplitude spectrum, the non-DC components are scaled by a factor of two. The spectrum implementation also uses the window sum for coherent-gain correction.

4.2.4 Windowing

The analyzer supports four windows:

- Rectangular.
- Hann.
- Hamming.
- Blackman.

For an input signal $x[n]$ and window $w[n]$:

$$x_w[n] = x[n]w[n] \quad (4.13)$$

Windowing reduces spectral leakage caused by finite-length signal observation.

4.2.5 Short-Time Fourier Transform

The STFT divides the signal into overlapping frames and computes the FFT of each windowed frame:

$$X(m, k) = \sum_{n=0}^{N-1} x[n + mH]w[n]e^{-j2\pi kn/N} \quad (4.14)$$

where H is the hop size.

The implemented configuration is:

- Window size: 1024 samples.
- Hop size: 256 samples.
- Default sampling rate: 44.1 kHz.

Therefore:

$$\Delta f = \frac{44100}{1024} \approx 43.07 \text{ Hz} \quad (4.15)$$

and

$$\Delta t = \frac{256}{44100} \approx 5.80 \text{ ms} \quad (4.16)$$

The spectrogram displays the magnitude of each STFT frame in decibels.

4.3 Theoretical Background

The project uses the following DSP concepts:

- Sampling and Nyquist frequency.
- Discrete-time signal representation.
- RMS and signal statistics.
- DFT and FFT.
- Frequency resolution.
- Spectral leakage.
- Window functions.
- Short-Time Fourier Transform.
- Time-frequency representation.

For a sampling rate F_s , the Nyquist frequency is:

$$f_N = \frac{F_s}{2} \quad (4.17)$$

For the default sampling rate:

$$f_N = 22.05 \text{ kHz} \quad (4.18)$$

Chapter 5

Implementation Details

5.1 Tools and Technologies

- **Programming Language:** JavaScript.
- **Frontend:** React.
- **Build Tool:** Vite.
- **Visualization:** HTML Canvas.
- **Audio:** Web Audio API.
- **Icons:** Lucide React.
- **Platform:** Web.
- **Hosting:** Vercel.

The DSP algorithms are implemented directly in JavaScript. No external DSP server is required.

5.2 System Architecture

The implementation is divided into four main areas.

5.2.1 Generator Module

`signalGenerator.js` contains the signal generation functions.

The generator accepts waveform parameters and returns a `Float32Array` of samples.

5.2.2 DSP Module

The `dsp` directory contains:

- `fft.js`: radix-2 FFT and power-of-two utilities.
- `spectrum.js`: windowed one-sided spectrum calculation.
- `stft.js`: STFT calculation.
- `windows.js`: window functions.
- `metrics.js`: RMS, peak, mean, sample count, and duration.
- `signalGenerator.js`: signal generation.

5.2.3 Audio Module

The audio directory contains:

- `audioDecoder.js`: local audio decoding and mono conversion.
- `audioPlayer.js`: browser playback and stop control.
- `wavExporter.js`: PCM16 WAV generation and download.

Uploaded audio retains its decoded sampling rate.

5.2.4 React Components

The interface is divided into:

- Generator components.
- Analyzer components.
- Common UI components.
- Layout components.

The analyzer contains separate components for the time-domain plot, frequency-domain plot, spectrogram, statistics, and analyzer metadata.

5.3 Key Algorithms / Pseudocode

5.3.1 Signal Generation

```
generateSignal(config):  
  read waveform parameters  
  calculate number of samples  
  create Float32Array  
  
  for each sample n:  
    calculate time t  
    evaluate selected waveform  
    multiply by amplitude  
    store sample  
  
  return samples
```

5.3.2 FFT Spectrum

```
computeSpectrum(samples, sampleRate, window):  
  choose next power-of-two FFT size  
  generate selected window  
  calculate window sum  
  multiply samples by window  
  compute FFT  
  keep positive-frequency bins  
  
  for each frequency bin:
```

```
    calculate magnitude
    apply one-sided scaling
    apply coherent-gain normalization
    calculate dB magnitude

    return frequencies and magnitudes
```

5.3.3 STFT

```
computeSTFT(samples, windowSize, hopSize):
    generate analysis window

    for each complete overlapping frame:
        apply window
        compute FFT
        keep positive-frequency bins
        convert magnitude to dB
        store frame

    calculate time and frequency axes
    return spectrogram data
```

5.4 User Interface

The application contains two primary panels.

5.4.1 Generator Panel

The Generator panel provides:

- Waveform selection.
- Frequency control.
- Start and end frequency controls for chirps.
- Duty-cycle control for square waves.
- Amplitude control.
- Duration control.
- Generate and Analyze.
- Audio upload.
- Play and Stop.
- WAV export.

5.4.2 Analyzer Panel

The Analyzer panel provides:

- Window selection.
- Sampling metadata.
- RMS, peak, and DC mean.
- Time-domain plot.
- Frequency-domain plot.
- STFT spectrogram.
- Pan control.
- Reset view.
- Fullscreen view.

The plots are rendered using HTML Canvas. The plot components use responsive resizing and support fullscreen viewing.

Chapter 6

Problems Faced and Solutions

6.1 Problem 1: Component Import Resolution

Issue: Initial component paths used inconsistent directory names for the analyzer components.

Solution: The project structure was standardized to use `src/components/analyzer/`. All imports were updated accordingly.

6.2 Problem 2: FFT and Spectrum Scaling

Issue: A direct FFT magnitude does not directly represent the amplitude of the original sinusoid. Windowing also changes the amplitude.

Solution: The spectrum implementation was changed to use window-sum coherent-gain correction and one-sided amplitude scaling.

6.3 Problem 3: Sampling Rate Handling

Issue: Generated signals use 44.1 kHz, while uploaded audio may use another sampling rate.

Solution: The sampling rate was propagated through the application state. Generated signals use 44.1 kHz and uploaded audio uses the decoded sampling rate. FFT, STFT, duration, Nyquist frequency, playback, and export use the active sampling rate.

6.4 Problem 4: Audio Playback and Export

Issue: Playback and export must use the currently analyzed signal, including uploaded audio.

Solution: The generator actions were connected to the shared sample array and active sampling rate. The same signal can therefore be played or exported after generation or upload.

6.5 Problem 5: Plot Readability

Issue: The initial plots contained basic axes and controls but did not provide a consistent technical visualization style.

Solution: The plots were redesigned with consistent axes, labels, grid lines, metadata, pan control, reset control, fullscreen support, and responsive Canvas rendering.

Chapter 7

Results and Discussion

7.1 Experimental Setup

Testing was performed using generated signals and local audio files.

The main generated-signal tests used:

- Sampling rate: 44.1 kHz.
- Duration: 1 s.
- Amplitude: 0.8.
- Sine frequency tests: 440 Hz and 1000 Hz.
- Square-wave duty cycle: 50%.
- Chirp: 200 Hz to 3000 Hz over 1 s.

The following functional cases were tested:

- Signal generation.
- Signal statistics.
- FFT frequency analysis.
- Window comparison.
- Chirp STFT analysis.
- Audio upload and sample-rate propagation.
- Playback and stop.
- WAV export.
- Pan, reset, and fullscreen controls.
- Responsive layout.

Signal	Amplitude	Expected RMS
Sine	0.8	0.5657
Square, 50% duty	0.8	0.8000
Triangle	0.8	0.4619

Table 7.1: Theoretical RMS values used for validation

7.2 Quantitative Results

The generated signals were compared with their theoretical RMS values.

For a 1 s generated signal at 44.1 kHz:

$$N = 44100 \quad (7.1)$$

For the 1 kHz FFT test, the implementation uses:

$$N = 65536 \quad (7.2)$$

giving:

$$\Delta f = \frac{44100}{65536} \approx 0.673 \text{ Hz} \quad (7.3)$$

The FFT peak was observed near the expected 1 kHz frequency. The small frequency offset is expected because 1000 Hz is not exactly an FFT-bin frequency for this FFT size.

7.3 Application Screenshots

The following screenshots document the implemented application interface and the main analysis modes. The image files can be placed inside the `screenshots/` directory using the filenames shown below.

7.3.1 Complete Application Interface

Figure 7.1 shows the complete Signal Lab interface. The Generator panel is shown alongside the Analyzer panel.

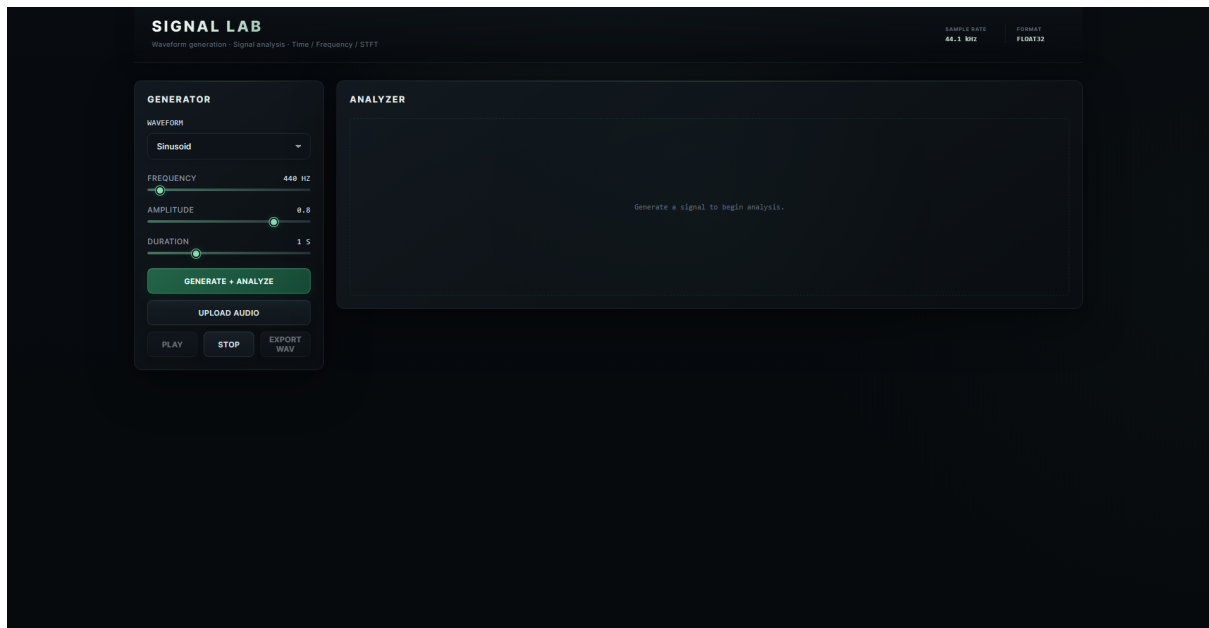


Figure 7.1: Complete Signal Lab application interface.

7.3.2 Signal Generation and Time-Domain Analysis

Figure 7.2 shows generated-signal controls together with the time-domain representation and signal statistics.

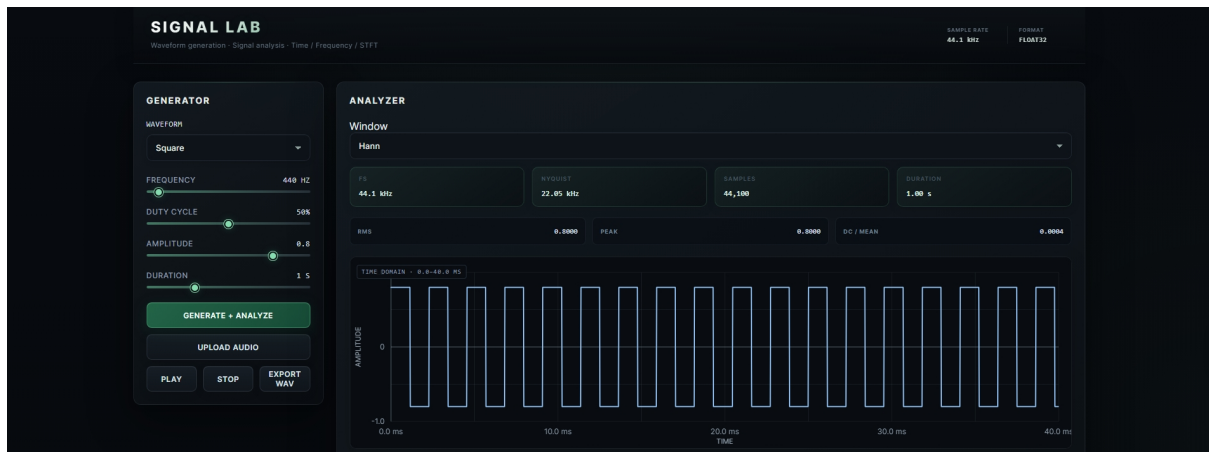


Figure 7.2: Signal generation and time-domain analysis.

7.3.3 Frequency-Domain Analysis

Figure 7.3 shows the FFT-based frequency-domain analysis of a generated signal.

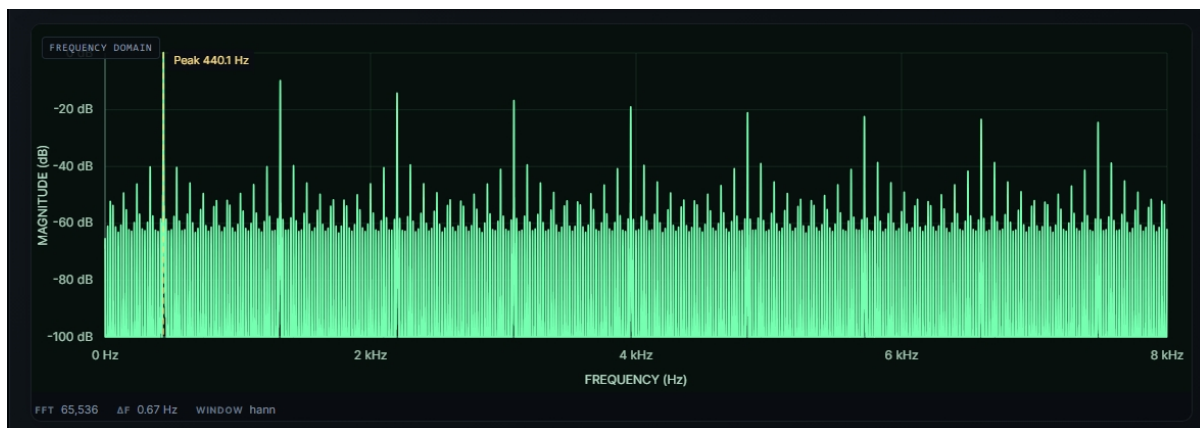


Figure 7.3: FFT-based frequency-domain analysis.

7.3.4 Windowing Analysis

Figure 7.4 shows the frequency-domain analyzer with a selected analysis window. The application supports Rectangular, Hann, Hamming, and Blackman windows.



Figure 7.4: Frequency-domain analysis with window selection.

7.3.5 Chirp and STFT Analysis

Figure 7.5 shows the STFT representation of the 200 Hz to 3000 Hz chirp.



Figure 7.5: STFT spectrogram of a 200 Hz to 3000 Hz chirp.

7.3.6 Audio Upload and Analysis

Figure 7.6 shows the application after loading an audio file. The analyzer uses the sampling rate obtained from the decoded audio.

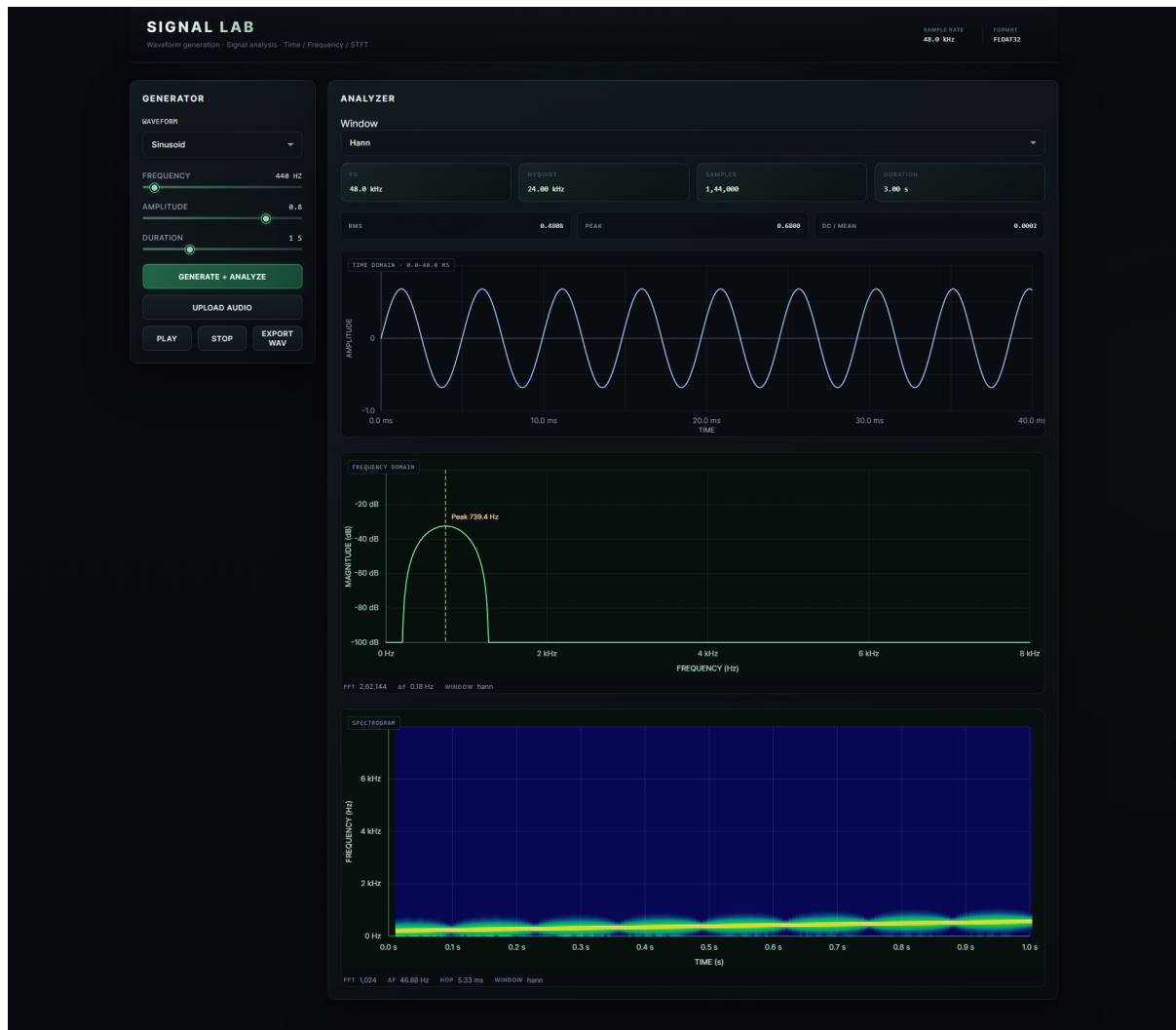


Figure 7.6: Uploaded audio analyzed using its decoded sampling rate.

7.3.7 Interactive Plot View

Figure 7.7 shows the fullscreen plot view with the interactive plot controls.

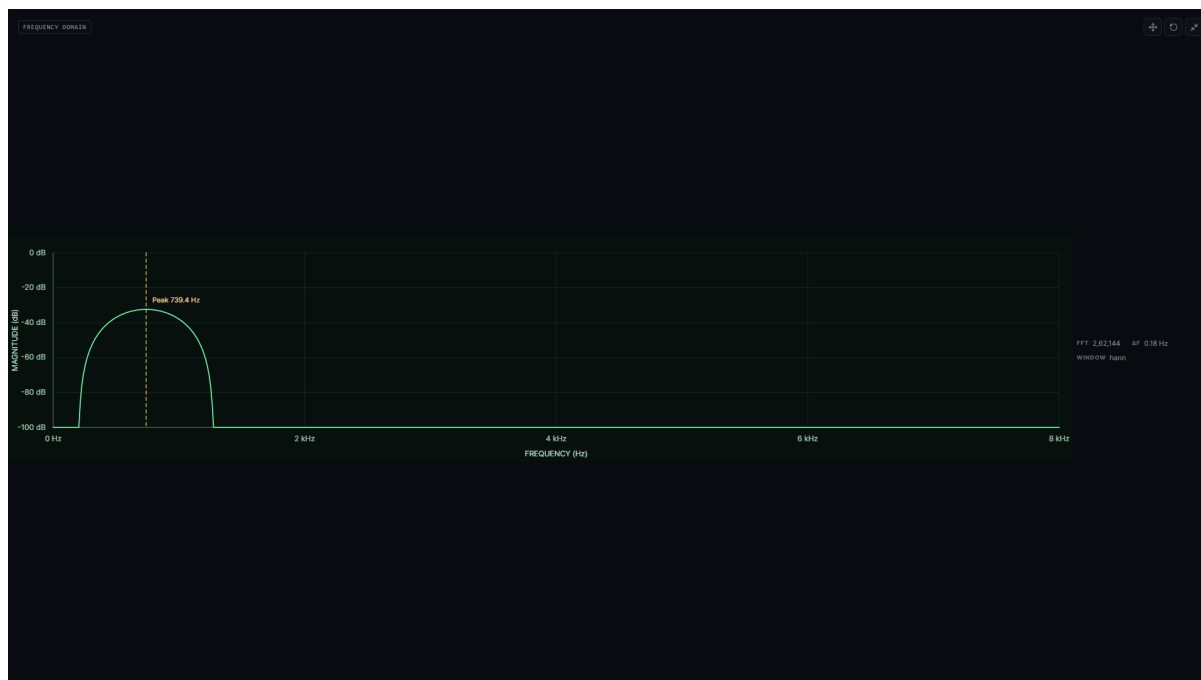


Figure 7.7: Fullscreen plot view with pan and reset controls.

7.4 Qualitative Results

7.4.1 Time-Domain Analysis

The generated sine, square, and triangle signals produced the expected waveform shapes.

The square wave changed according to the selected duty cycle. The triangle wave showed a linear rising and falling shape. The chirp showed a changing period over time.

7.4.2 Frequency-Domain Analysis

The frequency-domain analyzer produced a clear spectral peak for sinusoidal input.

Changing the analysis window changed the spectral leakage and main-lobe behavior. This allowed direct comparison of Rectangular, Hann, Hamming, and Blackman windows.

7.4.3 STFT Analysis

The chirp test used a sweep from 200 Hz to 3000 Hz over 1 s. The spectrogram showed a diagonal frequency trajectory from the lower frequency region toward the higher frequency region.

This demonstrates the advantage of STFT for time-varying signals. A single FFT gives the overall frequency content, while STFT shows how the frequency content changes with time.

7.4.4 Audio Analysis

Uploaded audio was decoded in the browser and analyzed using its actual sampling rate. The displayed sampling rate and Nyquist frequency changed accordingly.

Playback and WAV export operated on the currently active signal.

7.5 Discussion

The implementation satisfies the main functional requirements of the project.

The signal generation results follow the expected mathematical properties. The FFT provides the required frequency-domain analysis. Window selection demonstrates spectral leakage behavior. The STFT provides time-frequency analysis for non-stationary signals.

The implementation is client-side. This keeps the system simple and avoids the need for a backend DSP service.

The current implementation does not include hardware acquisition, filter design, real-time microphone processing, or advanced DSP algorithms. These were outside the implemented project scope.

Chapter 8

Conclusions

8.1 Summary of Contributions

Signal Lab was implemented as a complete web-based signal generator and analyzer.

The system supports signal generation, signal statistics, FFT analysis, windowing, STFT analysis, audio upload, playback, and WAV export. The implementation was validated using known signal properties and time-frequency tests.

8.2 Limitations

- Generated signals use a fixed default sampling rate of 44.1 kHz.
- STFT uses a fixed 1024-sample window and 256-sample hop.
- No FIR or IIR filter module is implemented.
- No live microphone acquisition is implemented.
- The STFT visualization excludes incomplete trailing frames.
- The application is designed primarily for educational DSP analysis.

8.3 Future Work

Possible extensions include:

- Configurable sampling rate.
- Configurable FFT and STFT parameters.
- Real-time microphone input.
- FIR and IIR filter design and analysis.
- Additional DSP measurements.
- Extended signal-generation options.

Chapter 9

Individual Contribution

This is an individual project. All work was carried out by the author.

- **Signal generation:** Implemented sine, square, triangle, chirp, and sinc signal generation.
- **DSP pipeline:** Implemented FFT, windowing, spectral analysis, STFT, and signal statistics.
- **Audio processing:** Implemented audio decoding, mono conversion, browser playback, and WAV export.
- **UI:** Designed and implemented the React interface and Canvas-based plots.
- **Testing:** Tested generated signals, frequency analysis, windowing, STFT, audio upload, playback, export, and responsive behavior.
- **Deployment:** Prepared and deployed the web application using Vercel.
- **Documentation:** Prepared the project documentation and final report.

Chapter 10

Annexure

Important: Access Permissions

All linked files should have “Anyone with the link” access where required for evaluation.

10.1 Video Demonstration

Video Link: <https://drive.google.com/file/d/1xML6Q-Kb9TRMUteVKyVFtg5CZrtheYCw/view?usp=sharing>

The demonstration covers:

- Signal generation.
- Time-domain analysis.
- Frequency-domain analysis.
- Window selection.
- STFT spectrogram.
- Audio upload.
- Playback and WAV export.

10.2 Source Code

GitHub Repository: <https://github.com/anjishnusaw91/Sig-Sys-IITM-Project>

The repository contains:

- Source code.
- README.md.
- Project configuration.
- DSP modules.
- Audio modules.
- React components.

10.3 Live Deployment

App URL: <https://signaanalyser.vercel.app/>

References

1. A. V. Oppenheim and R. W. Schaffer, *Discrete-Time Signal Processing*, 3rd ed., Pearson, 2010.
2. J. G. Proakis and D. G. Manolakis, *Digital Signal Processing: Principles, Algorithms, and Applications*, 4th ed., Pearson, 2007.
3. M. H. Hayes, *Schaum's Outline of Digital Signal Processing*, McGraw-Hill.
4. MDN Web Docs, *Web Audio API*. https://developer.mozilla.org/en-US/docs/Web/API/Web_Audio_API
5. React Documentation. <https://react.dev/>
6. Vite Documentation. <https://vite.dev/>